

Unico Implantação

Data do relatório	03/09/2026
Escopo auditado	Painel Next.js (app/, components/, features/, lib/, hooks/) e API Express/Prisma (server/src) — código-fonte completo do repositório, configs de deploy (docker-compose.yml, .github/workflows) e histórico do git.
Metodologia	Revisão manual, arquivo por arquivo, das 5 categorias solicitadas (isolamento de posse, permissão client-side, IDOR, segredos expostos, XSS), mapeadas para a stack detectada — ver nota metodológica abaixo.
Autor	Auditoria assistida por IA (Claude Code) — achados verificados em código real, sem execução de exploits contra ambiente vivo.

Nota metodológica — mapeamento para a stack

Stack detectada: monorepo com painel Next.js 16 / React 19 (app router) consumindo uma API própria em Express + TypeScript (server/), Prisma + PostgreSQL como ORM/banco, Redis + BullMQ para filas de deploy, autenticação por JWT em cookie httpOnly (própria, sem Supabase/RLS). Mapeamento das 5 categorias para esta stack: (1) "banco sem tranca" → não há RLS (não é Supabase); o isolamento é feito manualmente por código via implantationAccessWhere() em lib/access-control.ts, aplicado a cada query Prisma sensível — auditado logo abaixo. (2) "permissão no navegador" → checagem de AdminRole (ADMIN/MEMBER) feita tanto no middleware Express (requireAdmin) quanto, quando aplicável, em Server Components do Next.js — cruzado com a UI que esconde itens do sidebar por role. (3) IDOR → todo handler de rota Express com :id foi percorrido individualmente (não por amostragem). (4) Chaves expostas → grep no código-fonte rastreado pelo git, histórico do git, docker-compose.yml, workflows do GitHub Actions e arquivos de config; .env não versionados foram lidos apenas para confirmar que não vazam para o repositório. (5) XSS → sinks de HTML não escapado (dangerouslySetInnerHTML) e uso de href/src com dado dinâmico no frontend Next.js; não há template de e-mail nem renderização HTML no backend Express (API é JSON-only).

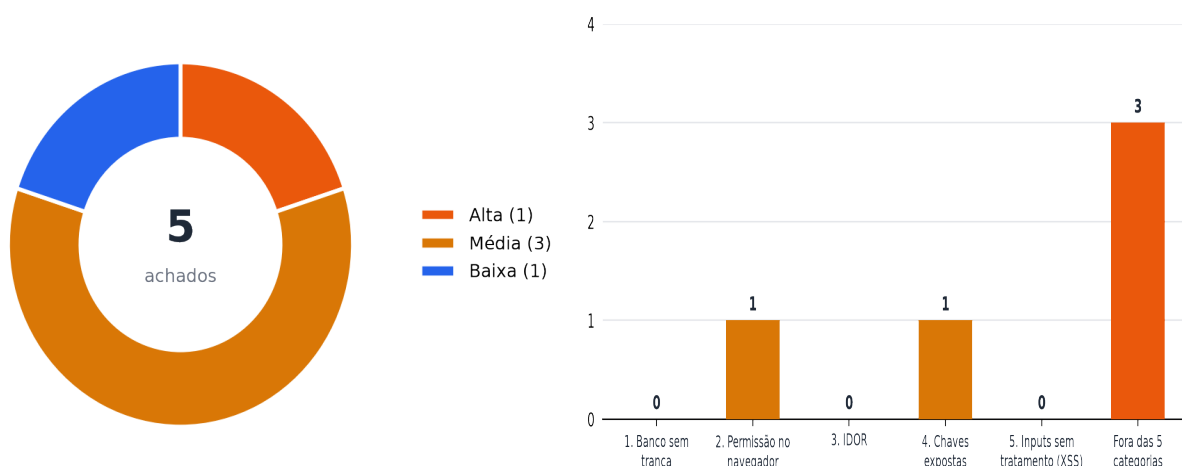
Resumo executivo



5 achados verificados no total. Nenhum achado crítico ou alta severidade nas 5 categorias centrais do escopo — o único item de severidade alta encontrado (CSRF) está fora delas, listado à parte na seção de achados adicionais.

✓ **Status: 5/5 achados corrigidos** — todas as correções abaixo já foram aplicadas ao código nesta mesma revisão (ver selo "CORRIGIDO" e o bloco "Correção aplicada" em cada achado detalhado).

Achados por severidade · Achados por categoria (cor = maior severidade)



Pontos fortes (verificados e corretos)

✓ **Isolamento por posse (categoria 1) aplicado de ponta a ponta**
lib/access-control.ts define `implantationAccessWhere(user)` — ADMIN vê tudo, MEMBER só `responsibleUserId = si mesmo`. Usado consistentemente em `implantation.service.ts` (list, stats, getById, activity), `review.service.ts` (findImplantationWithOnboarding) e `deployment.service.ts` (getLatestRun). Todas as rotas de escrita (update, cancel, approve, retryJob, activity, contact-import) chamam `getById()/findImplantationWithOnboarding()` primeiro, herdando o filtro — nenhum caminho encontrado que leia ou altere uma implantação sem passar por ele.

✓ **Backend revalida papel em toda rota sensível (categoria 2)**
`requireAdmin` em `users.routes.ts` (criar/editar/reset de senha/excluir) e em `audit-log.routes.ts`. As páginas Next.js `/admin/users` e `/admin/audit-logs` também reverificam `me.role === "ADMIN"` no servidor antes de buscar dados — a UI do sidebar esconde por conveniência, mas não é a única barreira.

**IDOR (categoria 3): nenhum handler encontrado sem checagem de posse**

Varredura de todos os *.routes.ts e *.controller.ts do backend: toda busca por :id passa por findFirst com implantationAccessWhere ou é ADMIN-only. Upload/download de importação de contatos usa nomes de arquivo gerados por randomUUID() (contact-import.service.ts) — sem travessia de diretório possível mesmo que o nome original seja hostil, pois é sempre path.basename()'d e nunca usado como caminho de leitura.

**Sem segredos hardcoded nem defaults inseguros (categoria 4)**

config/env.ts valida com zod e falha o boot se JWT_SECRET, CREDENTIALS_ENCRYPTION_KEY ou DATABASE_URL estiverem ausentes — sem fallback tipo `?? "default"`. .env e .env.local estão no .gitignore e confirmadamente nunca foram commitados (git log/ls-files vazios). docker-compose.yml exige POSTGRES_PASSWORD/REDIS_PASSWORD via \${VAR:?erro} — não sobe sem senha explícita. CI/CD (.github/workflows) usa exclusivamente GitHub Secrets, nunca grava segredo em log ou no repositório (script usa `umask 077` ao escrever server/.env).

**Sem XSS: toda entrada não confiável passa por interpolação JSX (categoria 5)**

As respostas do onboarding público (não autenticado, origem menos confiável) são renderizadas em OnboardingReview.tsx e ReviewEditor.tsx exclusivamente como filhos de JSX ({value}), que o React escapa por padrão. Único dangerouslySetInnerHTML do projeto é components/ui/chart.tsx, que injeta apenas nomes de cores de configuração estática do próprio código (shadcn), nunca dado de usuário. instanceBaseUrl usado em href é sempre reconstruído como https://<subdomínio-validado>.atenderbem.com (instance-url.ts), impedindo esquemas javascript: mesmo vindo de um campo de texto livre.

Pontos fracos — riscos centrais (já corrigidos)

- MEMBER podia reatribuir a posse de uma implantação (responsibleUserId) sem checagem de papel — a única lacuna encontrada no modelo de permissões, que é ADMIN-only em todo o resto do backend. Corrigido: agora exige ADMIN.
- Uma senha padrão fixa no código provisionava contas reais de atendimento no Atender Bem do cliente sempre que o onboarding não definia senha própria. Corrigido: senha gerada aleatoriamente por execução.
- Fora do escopo das 5 categorias, mas relevante: ausência de proteção CSRF (cookie SameSite=None sem token) e de rate limiting no login. Corrigido: verificação de Origin/Referer e rate limit adicionados.

Achados detalhados por categoria

Categoria 1 — Banco sem tranca (isolamento de posse/tenant)

Nenhum achado explorável nesta categoria — cobertura auditada na seção de pontos fortes.

Categoria 2 — Permissão definida no navegador

Sev.	Arquivo:linha	Descrição
MÉDIA	server/src/modules/implantations/implantation.schema.ts	PATCH /implantations/:id permite reatribuir responsibleUserId sem checagem de papel updateImplantationSchema herda responsibleUserId de createImplantationSchema e não o omite (só omite instanceUrl/planId/quotas). Em implantation.service.ts (update, linhas 211-221) o valor enviado pelo cliente é gravado sem nenhuma verificação de AdminRole — diferente de outras operações do mesmo módulo (quotas só na criação, implanterId auditado à parte) e do padrão do restante do backend, onde toda operação que reatribui posse de um recurso é ADMIN-only (ver users.routes.ts).
CORRIGIDO	:21, 30-39	

Por que é explorável: Qualquer sessão MEMBER que já tenha acesso a uma implantação (a sua própria, via `implantationAccessWhere`) pode, com um único PATCH, transferir a posse dela para o id de qualquer outro AdminUser — inclusive sem o consentimento do destinatário — bastando conhecer/adivinhar um id de usuário (visível hoje em GET /users, liberado a qualquer sessão autenticada). O efeito é uma escrita de controle de acesso que deveria ser privilégio de ADMIN, mas está liberada para qualquer papel.

Evidência:

```
export const updateImplantationSchema = createImplantationSchema
  .omit({ instanceUrl: true, planId: true, agentQuota: true, supervisorQuota: true,
    adminQuota: true })
  .partial()
  .extend({ implanterId: z.string().nullable().optional() });
// responsibleUserId NÃO está na lista de omit – continua editável por qualquer papel.
```

Impacto: Reatribuição não autorizada de posse de implantações entre contas do painel (MEMBER→qualquer).

Correção sugerida: Restringir a chave `responsibleUserId` no PATCH a sessões ADMIN (checagem explícita no controller/service, no mesmo padrão de `requireAdmin` usado em `users.routes.ts`), ou removê-la do `updateImplantationSchema` e criar uma rota dedicada de reatribuição.

✓ **Correção aplicada:** `implantation.service.ts` (update) agora lança `ForbiddenError` (403) quando `responsibleUserId` é enviado por um actor com role diferente de ADMIN, antes de qualquer escrita no banco. ADMIN continua reatribuindo normalmente.

Categoria 3 — IDOR

Nenhum achado de IDOR isolado — o único caso de posse mal controlada foi classificado na categoria 2 (é uma falha de checagem de papel, não de busca por id sem filtro).

Categoria 4 — Chaves expostas / credenciais padrão

Sev.	Arquivo:linha	Descrição
MÉDIA	server/src/jobs/processors/create-users.ts	Senha padrão fixa no código-fonte para todos os usuários do Atender Bem provisionados UNICO_DEFAULT_PASSWORD = "Unico@2026" é usada como senha de todo usuário criado na instância do cliente sempre que o onboarding não define uma senha padrão customizada (<code>usesCustomDefaultPassword=false</code> , o padrão do schema). O valor é idêntico em toda implantação e está em texto plano no repositório.
CORRIGIDO	:33	

Por que é explorável: Qualquer pessoa com acesso ao código-fonte (histórico de commits, contratados, ex-funcionários) conhece a senha inicial de todo atendente/supervisor/administrador criado em qualquer instância de cliente que não tenha customizado a senha padrão no onboarding — um alvo de credential-stuffing/força bruta trivial contra o Atender Bem do cliente até que cada usuário troque a própria senha.

Evidência:

```
// Decisão de produto: se o cliente não definir uma senha padrão própria no
// onboarding, todo novo usuário nasce com esta senha e deve trocá-la depois.
const UNICO_DEFAULT_PASSWORD = "Unico@2026";
```

Impacto: Acesso não autorizado a contas de atendimento do cliente na instância Atender Bem provisionada.

Correção sugerida: Gerar uma senha aleatória por usuário (ou por implantação) em vez de uma constante global, entregá-la por um canal fora do código-fonte (ex.: exibida uma única vez para o implantador, ou enviada ao responsável do cliente), e forçar troca no primeiro login se o Atender Bem suportar esse flag.

✓ **Correção aplicada:** create-users.ts agora gera uma senha aleatória forte por execução (generateDefaultPassword, 12 caracteres com maiúscula/minúscula/número/símbolo garantidos) em vez da constante fixa. A senha gerada só é devolvida no metadata da etapa (aba Atividade) quando de fato foi usada para criar algum usuário nesta execução — visível apenas a quem já tem acesso à implantação.

Categoria 5 — Inputs sem tratamento (XSS)

Nenhum achado de XSS — toda saída dinâmica passa por escaping automático do React; ver pontos fortes.

Achados adicionais (fora das 5 categorias solicitadas, verificados durante a auditoria)

Sev.	Arquivo:linha	Descrição
ALTA	server/src/lib/auth.ts; server/src/app.ts	Ausência de proteção CSRF em endpoints de escrita com cookie SameSite=None
CORRIGIDO	:46-52 (auth.ts); 20 (app.ts)	O cookie de sessão da API (unico_admin_session) é emitido com SameSite="none" + Secure por design (painel e API rodam em origens diferentes — ver comentário em auth.ts). O CORS (app.ts) restringe a origem para leitura de resposta, mas não impede o disparo da requisição em si: métodos; content-types 'simples' (POST sem preflight) chegam ao Express com o cookie anexado mesmo vindos de um site de terceiros. Não há CSRF token, verificação de Origin/Referer, nem SameSite=Lax/Strict como mitigação alternativa.

Por que é explorável: Endpoints POST que não exigem corpo JSON específico — POST `/implantations/:id/cancel`, POST `/implantations/:id/approve`, POST `/implantations/:id/onboarding-token/rotate`, POST `/deployments/:id/jobs/:type/retry`, POST `/auth/logout` — podem ser disparados por um formulário HTML autoenviado em qualquer site, usando o cookie `SameSite=None` da vítima já autenticada no painel. Rotas que exigem corpo JSON estrito (ex.: criação de usuário) não são exploráveis da mesma forma porque `express.json()` só interpreta `Content-Type application/json`, que um `<form>` comum não envia.

Evidência:

```
export const SESSION_COOKIE_OPTIONS = {
  httpOnly: true,
  secure: true,
  sameSite: "none" as const,
  ...
};
// app.ts: app.use(cors({ origin: env.FRONTEND_URL, credentials: true }));
// CORS restringe LEITURA da resposta, não o disparo da requisição.
```

Impacto: Cancelamento/aprovação de implantações, rotação de token de onboarding ou logout forçado sem ação intencional do usuário autenticado.

Correção sugerida: Adicionar um token CSRF (double-submit cookie ou header customizado verificado no servidor) nas rotas de escrita, ou validar o header `Origin/Referer` contra `FRONTEND_URL` antes de processar métodos mutantes.

✓ **Correção aplicada:** Novo middleware `requireTrustedOrigin` (`server/src/middleware/csrf.middleware.ts`), aplicado globalmente em `app.ts` antes de todas as rotas: para todo método que não seja `GET/HEAD/OPTIONS`, exige que o header `Origin` (ou `Referer` como fallback) bata exatamente com `FRONTEND_URL`, rejeitando com 403 caso contrário. O painel `Next.js` sempre envia `Origin` em `fetch cross-origin`, então o fluxo legítimo não é afetado.

MÉDIA

`server/src/modules/auth/auth.routes.ts`; `server/src/app.ts`

CORRIGIDO

:9 (`auth.routes.ts`)

POST `/auth/login` sem rate limiting nem bloqueio por tentativas

Não há `express-rate-limit`, `slow-down` ou qualquer contador de tentativas no `app.ts` ou nas rotas de `auth`. `package.json` do `server` não lista nenhuma dependência de `rate limiting`.

Por que é explorável: Um atacante pode tentar senhas indefinidamente contra qualquer e-mail de AdminUser conhecido (ex.: admin@unicocontato.com.br, visível no seed) sem qualquer atraso, CAPTCHA ou bloqueio, facilitando força bruta/credential stuffing contra o painel administrativo.

Evidência:

```
authRoutes.post("/login", asyncHandler(authController.login));  
// nenhum middleware de limitação antes deste handler
```

Impacto: Comprometimento de conta do painel administrativo por força bruta de senha.

Correção sugerida: Adicionar rate limiting por IP/e-mail (ex.: express-rate-limit) e um atraso ou bloqueio temporário após N tentativas falhas consecutivas para a mesma conta.

✓ **Correção aplicada:** Dependência express-rate-limit adicionada ao server/. POST /auth/login agora passa por loginRateLimit (auth.routes.ts): 10 tentativas por IP a cada 15 minutos, retornando 429 acima do limite.

BAIXA

features/implantations/components/ImplantationsTable.tsx; app/admin/implantations/[id]/page.tsx:135 (ImplantationsTable.tsx); 85, 95 (page.tsx)

CORRIGIDO

Links target="_blank" sem rel="noopener noreferrer"

Os links para a instância do cliente e para o link de onboarding abrem em nova aba (target="_blank") sem rel="noopener noreferrer". O next/link do Next 16 não adiciona esse atributo automaticamente.

Por que é explorável: A aba aberta (instanceBaseUrl é sempre https://<subdomínio>.atenderbem.com, validado pelo backend — risco baixo aqui, mas é o padrão a corrigir) mantém acesso via window.opener à página original, permitindo em tese redirecionar a aba de origem (reverse tabnabbing) caso o destino seja malicioso.

Evidência:

```
render={<Link href={implantation.instanceBaseUrl} target="_blank" />}
```

Impacto: Baixo neste caso concreto (destino validado), mas é uma prática insegura a padronizar.

Correção sugerida: Adicionar rel="noopener noreferrer" em todo target="_blank".

✓ **Correção aplicada:** rel="noopener noreferrer" adicionado aos três links target="_blank" identificados (ImplantationsTable.tsx e app/admin/implantations/[id]/page.tsx). Grep confirmou não haver mais nenhum target="_blank" sem o atributo no projeto.

Recomendações priorizadas — todas aplicadas

Prior.	Ação	Achado	Status
P1	Restringir a alteração de responsibleUserId em PATCH /implantations/:id a sessões ADMIN.	F1	✓ Corrigido
P1	Adicionar proteção CSRF (token ou verificação de Origin) nas rotas de escrita da API.	F3	✓ Corrigido
P2	Substituir a senha padrão fixa de usuários provisionados por um valor gerado por implantação/usuário.	F2	✓ Corrigido
P2	Adicionar rate limiting e bloqueio por tentativas em POST /auth/login.	F4	✓ Corrigido
P3	Adicionar rel="noopener noreferrer" em todos os links target="_blank".	F5	✓ Corrigido

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de cada issue.

```
--- ISSUE 1 ---
### [Segurança] PATCH /implantations/:id permite reatribuir responsibleUserId sem checagem de papel

**Labels:** security, media

**Descrição do problema**
updateImplantationSchema herda responsibleUserId de createImplantationSchema e não o omite (só omite instanceUrl/planId/quotas). Em implantation.service.ts (update, linhas 211-221) o valor enviado pelo cliente é gravado sem nenhuma verificação de AdminRole – diferente de outras operações do mesmo módulo (quotas só na criação, implanterId auditado à parte) e do padrão do restante do backend, onde toda operação que reatribui posse de um recurso é ADMIN-only (ver users.routes.ts). Qualquer sessão MEMBER que já tenha acesso a uma implantação (a sua própria, via implantationAccessWhere) pode, com um único PATCH, transferir a posse dela para o id de qualquer outro AdminUser – inclusive sem o consentimento do destinatário – bastando conhecer/adivinhar um id de usuário (visível hoje em GET /users, liberado a qualquer sessão autenticada). O efeito é uma escrita de controle de acesso que deveria ser privilégio de ADMIN, mas está liberada para qualquer papel.
Arquivo: `server/src/modules/implantations/implantation.schema.ts:21, 30-39`

**Evidência**

server/src/modules/implantations/implantation.schema.ts:21, 30-39

export const updateImplantationSchema = createImplantationSchema .omit({ instanceUrl: true, planId: true, agentQuota: true, supervisorQuota: true, adminQuota: true })
.partial() .extend({ implanterId: z.string().nullable().optional() }); //
responsibleUserId NÃO está na lista de omit – continua editável por qualquer papel.

**Impacto**
Reatribuição não autorizada de posse de implantações entre contas do painel (MEMBER→qualquer).

**Sugestão de correção**
Restringir a chave responsibleUserId no PATCH a sessões ADMIN (checagem explícita no controller/service, no mesmo padrão de requireAdmin usado em users.routes.ts), ou removê-la do updateImplantationSchema e criar uma rota dedicada de reatribuição.

**Critérios de aceite**
- [ ] MEMBER autenticado recebe 403 ao enviar responsibleUserId em PATCH /implantations/:id
- [ ] ADMIN continua conseguindo reatribuir normalmente
- [ ] Mudança coberta por teste automatizado (ou manual documentado) e registrada em audit-log

--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---  
### [Segurança] Senha padrão fixa no código-fonte para todos os usuários do Atender Bem provisionados  
  
**Labels:** security, media  
  
**Descrição do problema**  
UNICO_DEFAULT_PASSWORD = "Unico@2026" é usada como senha de todo usuário criado na instância do cliente sempre que o onboarding não define uma senha padrão customizada (usesCustomDefaultPassword=false, o padrão do schema). O valor é idêntico em toda implantação e está em texto plano no repositório. Qualquer pessoa com acesso ao código-fonte (histórico de commits, contratados, ex-funcionários) conhece a senha inicial de todo atendente/supervisor/administrador criado em qualquer instância de cliente que não tenha customizado a senha padrão no onboarding – um alvo de credential-stuffing/força bruta trivial contra o Atender Bem do cliente até que cada usuário troque a própria senha. Arquivo: `server/src/jobs/processors/create-users.ts:33`  
  
**Evidência**  
  
server/src/jobs/processors/create-users.ts:33  
  
// Decisão de produto: se o cliente não definir uma senha padrão própria no // onboarding, todo novo usuário nasce com esta senha e deve trocá-la depois. const  
UNICO_DEFAULT_PASSWORD = "Unico@2026";  
  
**Impacto**  
Acesso não autorizado a contas de atendimento do cliente na instância Atender Bem provisionada.  
  
**Sugestão de correção**  
Gerar uma senha aleatória por usuário (ou por implantação) em vez de uma constante global, entregá-la por um canal fora do código-fonte (ex.: exibida uma única vez para o implantador, ou enviada ao responsável do cliente), e forçar troca no primeiro login se o Atender Bem suportar esse flag.  
  
**Critérios de aceite**  
- [ ] Senha padrão deixa de ser uma constante idêntica entre implantações  
- [ ] Novo mecanismo documentado em docs/ e testado numa implantação de homologação  
- [ ] Segredo não aparece mais em texto plano no código-fonte  
  
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
### [Segurança] Ausência de proteção CSRF em endpoints de escrita com cookie SameSite=None

**Labels:** security, alta

**Descrição do problema**
O cookie de sessão da API (unico_admin_session) é emitido com SameSite="none" + Secure por design (painel e API rodam em origens diferentes – ver comentário em auth.ts). O CORS (app.ts) restringe a origem para leitura de resposta, mas não impede o disparo da requisição em si: métodos/; content-types 'simples' (POST sem preflight) chegam ao Express com o cookie anexado mesmo vindos de um site de terceiros. Não há CSRF token, verificação de Origin/Referer, nem SameSite=Lax/Strict como mitigação alternativa. Endpoints POST que não exigem corpo JSON específico – POST /implantations/:id/cancel, POST /implantations/:id/approve, POST /implantations/:id/onboarding-token/rotate, POST /deployments/:id/jobs/:type/retry, POST /auth/logout – podem ser disparados por um formulário HTML autoenviado em qualquer site, usando o cookie SameSite=None da vítima já autenticada no painel. Rotas que exigem corpo JSON estrito (ex.: criação de usuário) não são exploráveis da mesma forma porque express.json() só interpreta Content-Type application/json, que um <form> comum não envia. Arquivo: `server/src/lib/auth.ts; server/src/app.ts:46-52 (auth.ts); 20 (app.ts)`

**Evidência**

server/src/lib/auth.ts; server/src/app.ts:46-52 (auth.ts); 20 (app.ts)

export const SESSION_COOKIE_OPTIONS = { httpOnly: true, secure: true, sameSite: "none" as const, ... }; // app.ts: app.use(cors({ origin: env.FRONTEND_URL, credentials: true }));
// CORS restringe LEITURA da resposta, não o disparo da requisição.

**Impacto**
Cancelamento/aprovação de implantações, rotação de token de onboarding ou logout forçado sem ação intencional do usuário autenticado.

**Sugestão de correção**
Adicionar um token CSRF (double-submit cookie ou header customizado verificado no servidor) nas rotas de escrita, ou validar o header Origin/Referer contra FRONTEND_URL antes de processar métodos mutantes.

**Critérios de aceite**
- [ ] POST/PATCH/DELETE autenticados exigem CSRF token válido ou Origin confiável
- [ ] Requisição forjada de origem externa recebe 403
- [ ] Fluxo legítimo do painel Next.js continua funcionando sem fricção adicional para o usuário

--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
### [Segurança] POST /auth/login sem rate limiting nem bloqueio por tentativas

**Labels:** security, media

**Descrição do problema**
Não há express-rate-limit, slow-down ou qualquer contador de tentativas no app.ts ou nas rotas de auth. package.json do server não lista nenhuma dependência de rate limiting. Um atacante pode tentar senhas indefinidamente contra qualquer e-mail de AdminUser conhecido (ex.: admin@unicocontato.com.br, visível no seed) sem qualquer atraso, CAPTCHA ou bloqueio, facilitando força bruta/credential stuffing contra o painel administrativo. Arquivo: `server/src/modules/auth/auth.routes.ts; server/src/app.ts:9 (auth.routes.ts)`

**Evidência**

server/src/modules/auth/auth.routes.ts; server/src/app.ts:9 (auth.routes.ts)

authRoutes.post("/login", asyncHandler(authController.login)); // nenhum middleware de limitação antes deste handler

**Impacto**
Comprometimento de conta do painel administrativo por força bruta de senha.

**Sugestão de correção**
Adicionar rate limiting por IP/e-mail (ex.: express-rate-limit) e um atraso ou bloqueio temporário após N tentativas falhas consecutivas para a mesma conta.

**Critérios de aceite**
- [ ] Tentativas de login acima de um limite configurável retornam 429
- [ ] Bloqueio não impede logins legítimos em uso normal

--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---
### [Segurança] Links target="_blank" sem rel="noopener noreferrer"

**Labels:** security, baixa

**Descrição do problema**
Os links para a instância do cliente e para o link de onboarding abrem em nova aba (target="_blank") sem rel="noopener noreferrer". O next/link do Next 16 não adiciona esse atributo automaticamente. A aba aberta (instanceBaseUrl é sempre https://<subdomínio>.atenderbem.com, validado pelo backend – risco baixo aqui, mas é o padrão a corrigir) mantém acesso via window.opener à página original, permitindo em tese redirecionar a aba de origem (reverse tabnabbing) caso o destino seja malicioso. Arquivo: `features/implantations/components/ImplantationsTable.tsx; app/admin/implantations/[id]/page.tsx:135 (ImplantationsTable.tsx); 85, 95 (page.tsx)`

**Evidência**

features/implantations/components/ImplantationsTable.tsx;
app/admin/implantations/[id]/page.tsx:135 (ImplantationsTable.tsx); 85, 95 (page.tsx)

render={<Link href={implantation.instanceBaseUrl} target="_blank" />}

**Impacto**
Baixo neste caso concreto (destino validado), mas é uma prática insegura a padronizar.

**Sugestão de correção**
Adicionar rel="noopener noreferrer" em todo target="_blank".

**Critérios de aceite**
- [ ] Todo Link/<a> com target="_blank" no projeto tem rel="noopener noreferrer"

--- FIM ISSUE 5 ---
```